

Shopify Read-Only Commerce Intelligence & Alerting

Blueprint untuk dashboard analytics, anomaly detection, dan operational alert tanpa melakukan mutation/write ke Shopify.

Core constraint

Semua interaksi ke Shopify dibatasi pada GraphQL query/read. Sistem boleh menyimpan snapshot pada storage lokal untuk analisis, tetapi tidak mengubah product, order, inventory, customer, discount, metafield, atau resource Shopify lainnya.

Fokus	Commerce observability berbasis data Shopify
Output	Dashboard + alert feed + evidence/diagnosis
Stack	Python, HTTPX, Pandas, DuckDB, Streamlit, Plotly
MVP scopes	read_reports, read_orders, read_products, read_inventory
Non-goal	Auto-remediation dan seluruh Shopify mutation/write

Status dokumen: exploration / technical proposal

Basis API yang direkomendasikan: Shopify GraphQL Admin API 2026-07 (current stable per September 2026).

Ringkasan Eksekutif

Project ini mengeksplorasi bagaimana data Shopify dapat dipakai untuk membangun sistem observability yang tidak hanya menampilkan KPI, tetapi juga mendeteksi perubahan yang tidak normal dan menjelaskan kemungkinan penyebabnya. Batas desain utama adalah Shopify read-only: aplikasi tidak memiliki workflow yang mengubah data merchant.

Pertanyaan eksplorasi utama

"Can we detect and explain important commerce problems from Shopify using read-only access?" Fokusnya bukan sekadar menampilkan sales, melainkan mengubah data read-only menjadi sinyal operasional yang dapat dipahami.

Struktur Dokumen

- 1. Latar belakang dan konteks problem
- 2. Rumusan masalah, tujuan, dan batasan scope
- 3. Scope Shopify dan data yang digunakan
- 4. Library dan tools Python
- 5. Arsitektur dan flowchart sistem
- 6. Metode implementasi langkah demi langkah
- 7. Metode analytics dan anomaly detection
- 8. Struktur folder modular
- 9. Bayangan hasil/output dashboard dan alert
- 10. Kriteria keberhasilan, risiko, dan referensi

Prinsip Desain

- **Read-only first.** Tidak ada mutation method pada Shopify client dan tidak meminta write scope untuk MVP.
- **Least privilege.** Scope diaktifkan hanya jika benar-benar diperlukan oleh use case.
- **Explainable before ML.** Mulai dari baseline, persentase deviasi, dan z-score; ML opsional setelah data dan aturan stabil.
- **API-cost aware.** Query dipaginasi, dicache, dan membaca throttle status agar tidak boros GraphQL cost.
- **Modular.** Collection, storage, analytics, alert, dan dashboard dipisahkan agar mudah diuji dan diganti.

1. Latar Belakang

Merchant Shopify memiliki banyak data operasional - penjualan, order, product, inventory, returns, funnel, dan pembayaran. Tantangannya adalah data tersebut tersebar dalam resource dan metrik yang berbeda. Dashboard biasa dapat menjawab "apa yang terjadi", tetapi sering belum menjawab "mengapa ini terjadi" atau "apakah perubahan ini masih normal".

Exploration ini memanfaatkan akses client dan token Shopify untuk membaca data melalui GraphQL Admin API. Nilai project berasal dari pemrosesan setelah data dibaca: normalisasi, perhitungan metric, pembentukan baseline, deteksi anomali, korelasi antar-sinyal, dan penyajian evidence dalam dashboard/alert.

Real-world example

Revenue turun 35% pada hari ini. Sistem tidak berhenti pada angka -35%. Sistem memeriksa apakah sessions ikut turun, conversion rate turun, checkout completion berubah, top product stockout, atau return rate meningkat. Hasilnya adalah alert dengan bukti yang membantu investigasi.

Mengapa read-only tetap bernilai?

- Risiko integrasi lebih kecil karena aplikasi tidak mengubah state store.
- Cocok untuk exploration, monitoring, audit, reporting, dan decision support.
- Memaksa desain untuk menghasilkan value dari insight, bukan automasi mutation.
- Mudah dipresentasikan sebagai observability layer di atas commerce platform.

Problem statement awal

Signal tersebar	Sales, order, inventory, funnel, dan return tidak selalu dilihat sebagai satu cerita.
Baseline lemah	Merchant melihat angka saat ini tanpa konteks "normal" berdasarkan histori.
Alert terlalu sederhana	Threshold statis sering menghasilkan false positive atau tidak menjelaskan penyebab.
API memiliki constraint	GraphQL memakai calculated query cost sehingga collector perlu cost-aware.
Scope perlu disiplin	Read-only dan least privilege harus terlihat pada desain, bukan hanya dokumentasi.

2. Rumusan Masalah, Tujuan, dan Batasan

2.1 Rumusan Masalah

1. Bagaimana mengambil data commerce Shopify secara aman dengan akses query/read-only?
2. Bagaimana mengubah data mentah menjadi KPI yang konsisten dan mudah dibandingkan?
3. Bagaimana menentukan apakah perubahan revenue, conversion, inventory, atau returns masih normal?
4. Bagaimana menjelaskan kemungkinan penyebab suatu anomaly menggunakan beberapa signal Shopify?
5. Bagaimana menyajikan hasil sebagai dashboard dan alert yang actionable tanpa melakukan write ke Shopify?

2.2 Tujuan

- Membangun Shopify query client yang tidak mengekspos mutation API.
- Mengambil data analytics/order/product/inventory yang relevan terhadap problem.
- Membangun local analytical store untuk snapshot dan baseline historis.
- Menghasilkan KPI dan anomaly score yang explainable.
- Menyediakan dashboard dan alert feed sebagai demonstrasi hasil exploration.
- Mendokumentasikan API constraints, scope constraints, dan modular architecture yang dapat dikembangkan lebih lanjut.

2.3 Batasan Scope

Area	IN scope	OUT of scope
Shopify API	GraphQL queries, pagination, ShopifyQL bila tersedia	Semua mutation/write
Data	Orders, sales analytics, products, inventory; returns opsional	Customer PII enrichment pada MVP
Storage	DuckDB/local snapshots untuk analytics	Menulis balik snapshot ke Shopify
Alert	In-app alert feed dan severity	Auto-fix inventory/order/product
Analytics	Rules, baseline, z-score, correlation	ML kompleks sebelum baseline stabil

3. Scope Shopify dan Data yang Digunakan

Daftar scope yang disediakan menunjukkan resource read dan write yang tersedia. Untuk exploration ini, write scopes tidak diperlukan. Prioritas MVP adalah empat capability: analytics/reporting, orders, products, dan inventory. Returns dan payout dapat menjadi extension setelah MVP.

Scope	Data utama	Peran pada project	Status
read_reports	shopifyqlQuery	Sales/funnel/analytics table jika akses memenuhi requirement	MVP - preferred
read_orders	Order, fulfillment, transaction	Order trend, AOV, operational facts	MVP
read_products	Product, variant	Product context dan product performance	MVP
read_inventory	InventoryItem, InventoryLevel	Stock risk dan days-of-stock	MVP
read_returns	Return	Return spike / product quality signal	Phase 2
read_shopify_payments_payouts	Payout, balance transaction	Cash/payout monitoring	Phase 2
read_all_orders	Extended order history	History di luar default order window	Optional / permission

Important implementation check

Jangan mengasumsikan token sudah memiliki semua scope. Langkah pertama implementasi adalah membaca `currentAppInstallation.accessScopes` dan membentuk capability matrix. Dokumentasi Shopify saat ini juga menyebut `shopifyqlQuery` membutuhkan `read_reports` serta requirement protected customer data tertentu; karena itu ShopifyQL harus divalidasi sebelum dijadikan hard dependency.

Catatan tambahan: `read_all_orders` merupakan scope tambahan untuk mengakses history order yang lebih luas daripada window order default dan memerlukan permission khusus. Untuk anomaly MVP, histori beberapa minggu biasanya sudah memadai, sehingga scope ini tidak harus menjadi blocker.

4. Library dan Tools yang Digunakan

4.1 Python Libraries

Library	Fungsi	Alasan dipilih	Prioritas
httpx	GraphQL HTTP client	Connection pooling, timeout, clean client abstraction	Wajib
python-dotenv	Environment config	Memisahkan token/shop domain dari source code	Wajib
pydantic	Config dan schema validation	Response/config lebih eksplisit dan testable	Wajib
tenacity	Retry/backoff	Menangani network error dan throttling secara terkendali	Wajib
pandas	Data processing	Aggregation, rolling baseline, join, metric calculation	Wajib
duckdb	Local analytics database	SQL lokal, ringan, integrasi langsung dengan Pandas	Recommended
streamlit	Dashboard app	Cepat untuk exploration dan multi-page data app	Recommended
plotly	Interactive charts	Line, funnel, bar, hover, drill-down ringan	Recommended
pytest	Automated tests	Validasi formula, query guard, transform, anomaly rules	Recommended
scipy / scikit-learn	Advanced anomaly detection	Dipakai hanya bila rules/statistics dasar tidak cukup	Optional Phase 2

4.2 Development Tools

Tool	Peran	Catatan
Python 3.11+	Runtime	Versi modern, type hints, ekosistem data matang.
VS Code / PyCharm	IDE	Linting, debugger, test runner.
venv / uv	Environment	Isolasi dependency dan reproducible setup.
Git	Version control	Track exploration, rules, dan query changes.
Shopify Dev/Admin	API source	Store domain + access token + granted read scopes.

Tool	Peran	Catatan
Bruno/Postman (opsional)	API inspection	Menguji query secara manual sebelum masuk collector.

Minimal install

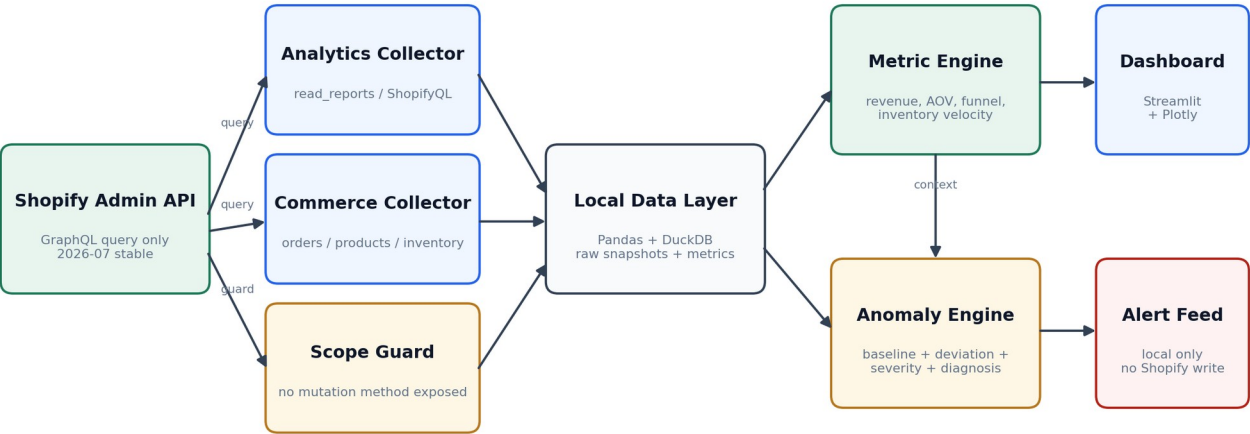
pip install httpx python-dotenv pydantic tenacity pandas duckdb streamlit plotly pytest

5. Arsitektur Sistem dan Method Flowchart

Arsitektur memisahkan lapisan Shopify access, collector, storage, analytics, anomaly, dan presentation. Pemisahan ini menjaga boundary read-only serta memudahkan unit testing pada setiap layer.

Read-only commerce observability architecture

All Shopify interactions are query-only. Local persistence is analytical storage, not store mutation.



Gambar 1. Arsitektur read-only commerce observability.

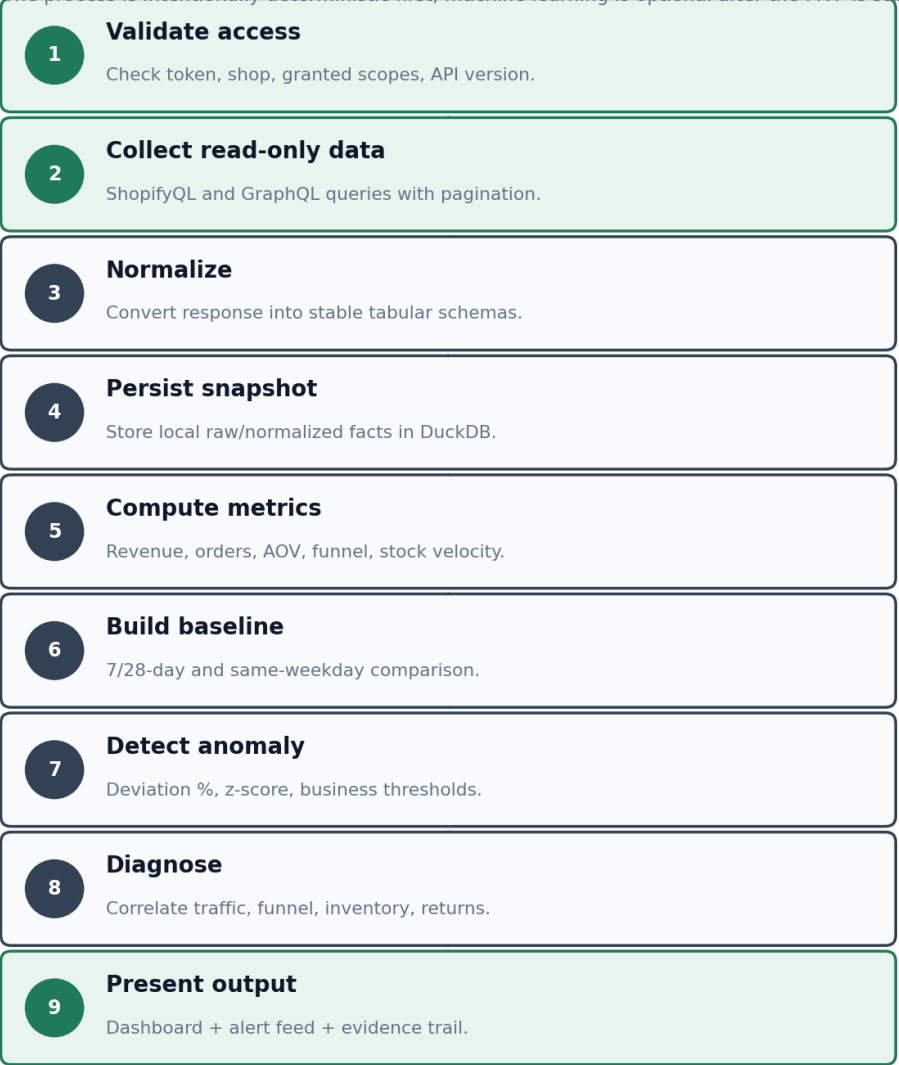
Read-only guard

ShopifyClient hanya menyediakan method query(). Tidak ada mutate(), create(), update(), delete(), inventoryAdjust(), atau fungsi write lain. Query documents disimpan di modul queries/ dan dapat diaudit melalui test.

5.2 Method Flowchart

Method flow: from access validation to explainable alert

The process is intentionally deterministic first; machine learning is optional after the MVP is stable.



Gambar 2. Alur metode implementasi dan analisis.

6. Metode Implementasi - Step by Step

Step 0 - Secure configuration

Simpan SHOPIFY_STORE, SHOPIFY_ACCESS_TOKEN, dan SHOPIFY_API_VERSION di .env. Tambahkan .env ke .gitignore. Gunakan .env.example tanpa credential.

Step 1 - Validate capability

Call query kecil untuk shop dan currentAppInstallation.accessScopes. Bentuk capability matrix dan hentikan collector yang scope-nya tidak tersedia.

Step 2 - Build query-only client

Gunakan httpx.Client dengan base URL GraphQL, default headers, timeout, error mapping, dan capture extensions.cost.throttleStatus.

Step 3 - Build collectors

Pisahkan analytics, orders, products, dan inventory. Terapkan cursor pagination. Collector hanya mengembalikan raw/normalized records.

Step 4 - Normalize schema

Ubah GraphQL edges/nodes dan ShopifyQL table rows menjadi DataFrame dengan nama kolom stabil, timezone konsisten, dan numeric type yang benar.

Step 5 - Persist local snapshot

Tulis snapshot ke DuckDB. Simpan collected_at agar sistem dapat membedakan state dari waktu ke waktu. Ini bukan write ke Shopify.

Step 6 - Compute metrics

Hitung revenue, orders, AOV, conversion/funnel jika tersedia, units sold, inventory velocity, days-of-stock, dan return rate bila scope tersedia.

Step 7 - Create baseline

Gunakan rolling 7/28 hari dan same-weekday reference. Baseline harus memisahkan current value dan expected value.

Step 8 - Detect and classify

Hitung deviation %, z-score, serta business rule. Gabungkan menjadi severity INFO/WARNING/CRITICAL.

Step 9 - Diagnose

Korelasikan revenue dengan sessions/conversion, checkout stage, inventory, serta returns. Alert harus membawa evidence, bukan hanya label anomaly.

Step 10 - Visualize and evaluate

Tampilkan KPI cards, trend chart, funnel evidence, inventory risk, dan alert feed. Uji dengan synthetic anomaly serta data real store.

API-cost strategy

- Prefer query yang hanya meminta field yang benar-benar dipakai.

- Gunakan cursor pagination dan batch size yang konservatif.
- Baca requestedQueryCost, actualQueryCost, currentlyAvailable, dan restoreRate dari response extension jika tersedia.
- Cache hasil di local store agar dashboard tidak selalu call Shopify ketika user me-refresh UI.
- Gunakan retry dengan exponential backoff untuk network/transient errors; jangan retry GraphQL validation error secara buta.

7. Metode Analytics dan Anomaly Detection

7.1 KPI utama

Metric	Contoh formula	Interpretasi
Revenue	<code>sum(total_sales)</code>	Nilai penjualan dalam window analisis.
Orders	<code>count(order)</code>	Volume transaksi.
AOV	<code>revenue / orders</code>	Rata-rata nilai per order.
Conversion rate	<code>completed_checkout / sessions</code>	Efektivitas funnel jika sessions tersedia.
Inventory velocity	<code>units_sold / days</code>	Kecepatan penjualan SKU/product.
Days of stock	<code>on_hand / avg_daily_units</code>	Perkiraan berapa hari stok tersisa.
Return rate	<code>returned_units / sold_units</code>	Sinyal kualitas/product fit jika returns tersedia.

7.2 Baseline dan severity

Untuk MVP, anomaly detection tidak perlu langsung memakai machine learning. Metode deterministik lebih mudah diuji dan dijelaskan. Gunakan dua jalur: statistical deviation dan business rule.

Metode	Rumus/aturan	Contoh trigger
Percent deviation	$(\text{current} - \text{baseline}) / \text{baseline} \times 100$	Revenue < -30% vs same-weekday baseline
Z-score	$(\text{current} - \text{mean}) / \text{std}$	$ z \geq 2$ warning; $ z \geq 3$ critical
Business threshold	Rule berdasarkan domain	Days of stock < 2 hari
Correlation diagnosis	Bandingkan beberapa signal	Traffic stable + conversion drop = funnel issue candidate

Recommended baseline

Bandingkan current period dengan rolling 28 hari serta hari yang sama pada beberapa minggu sebelumnya. Ini mengurangi false positive akibat weekday/weekend pattern dibanding hanya memakai average semua hari.

7.3 Contoh diagnosis rule

- **Revenue turun + sessions turun** -> indikasi acquisition/traffic problem.
- **Revenue turun + sessions stabil + conversion turun** -> indikasi funnel/conversion problem.
- **Add-to-cart normal + completed checkout turun** -> indikasi degradation di checkout/payment stage.
- **Top product sales tinggi + days-of-stock sangat rendah** -> inventory revenue risk.
- **Return rate naik pada product tertentu** -> quality/expectation mismatch candidate.

8. Struktur Folder Modular

Struktur berikut memisahkan infrastructure, domain logic, presentation, dan tests. Nama package dapat disesuaikan, tetapi dependency direction sebaiknya tetap: dashboard memanggil services/analytics; analytics tidak bergantung pada Streamlit.

```
shopify-store-pulse/  
|-- .env.example  
|-- .gitignore  
|-- pyproject.toml  
|-- README.md  
|-- data/  
|   |-- .gitkeep  
|-- scripts/  
|   |-- collect_once.py  
|   |-- inspect_scopes.py  
|-- src/  
|   |-- store_pulse/  
|   |   |-- config.py  
|   |   |-- shopify/  
|   |   |   |-- client.py  
|   |   |   |-- pagination.py  
|   |   |   |-- scopes.py  
|   |   |   |-- queries/  
|   |   |       |-- analytics.py  
|   |   |       |-- orders.py  
|   |   |       |-- products.py  
|   |   |       |-- inventory.py  
|   |-- collectors/  
|   |   |-- analytics.py  
|   |   |-- orders.py  
|   |   |-- products.py  
|   |   |-- inventory.py  
|   |-- storage/  
|   |   |-- duckdb.py  
|   |   |-- schema.py  
|   |-- analytics/  
|   |   |-- metrics.py  
|   |   |-- baseline.py  
|   |   |-- diagnosis.py  
|   |-- alerts/  
|   |   |-- models.py  
|   |   |-- rules.py  
|   |   |-- severity.py  
|   |-- dashboard/  
|   |-- app.py
```

```

|         | -- overview.py
|         | -- funnel.py
|         | -- inventory.py
-- tests/
| -- test_read_only_guard.py
| -- test_pagination.py
| -- test_metrics.py
| -- test_baseline.py
| -- test_alert_rules.py

```

Tanggung jawab tiap layer

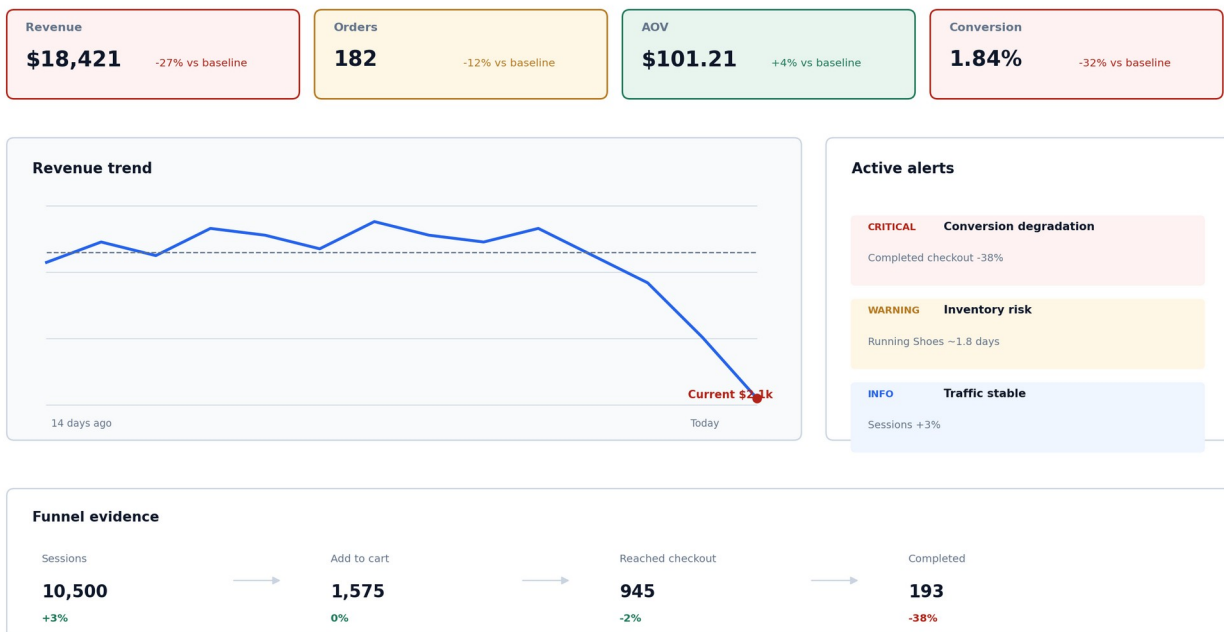
Layer	Tidak boleh melakukan	Harus menghasilkan
shopify/	Business metric logic	Safe GraphQL responses + API metadata
collectors/	UI rendering	Normalized records/DataFrames
storage/	Call Shopify API	Persist/read local analytical data
analytics/	Know Streamlit	Metrics, baseline, diagnosis
alerts/	Modify Shopify	Alert object + severity + evidence
dashboard/	Embed Shopify token in UI	Presentation dan filters

9. Bayangan Hasil / Output

Output utama adalah data product observability sederhana: satu halaman overview untuk health store, ditambah evidence yang menjelaskan anomaly. Angka pada mockup berikut bersifat synthetic untuk menggambarkan bentuk hasil, bukan data store aktual.

STORE PULSE

Mockup output - synthetic example data



Gambar 3. Mockup dashboard Store Pulse dengan synthetic example data.

Contoh bentuk alert

Field	Contoh nilai
Severity	CRITICAL
Title	Conversion degradation detected
Observed	Conversion 1.84%
Baseline	2.70% same-weekday baseline
Deviation	-32%
Evidence	Sessions +3%; add-to-cart 0%; completed checkout -38%
Likely cause	Checkout/funnel degradation candidate
Action	Investigate checkout/payment/channel behavior; no automatic Shopify change

10. Kriteria Keberhasilan dan Risiko

10.1 Definition of Done untuk Exploration

Area	Minimum success	Evidence
Access	Token berhasil query dan scope terdeteksi	Capability matrix
Read-only safety	Tidak ada mutation method/query	Unit test + code review
Data	Minimal 2-4 data sources masuk local model	Collector logs + DuckDB tables
Metrics	Revenue/order/AOV + 1 operational metric	Metric tests
Anomaly	Synthetic drop dapat terdeteksi	Test scenario
Diagnosis	Alert membawa minimal 2 supporting signals	Alert object
Dashboard	Overview dapat dibuka dan difilter	Streamlit UI
API behavior	Throttle/cost metadata ditangani	Client logs/test

10.2 Risiko dan Mitigasi

Risiko	Dampak	Mitigasi
Scope tidak granted	Collector tertentu gagal	Capability matrix + graceful feature disable
ShopifyQL access requirement	Funnel analytics mungkin tidak tersedia	Fallback ke order/product/inventory exploration
Rate limit/query cost	Data collection lambat/throttled	Cost-aware queries, cache, backoff
History pendek	Baseline kurang stabil	Gunakan available 4-8 weeks; synthetic test; optional read_all_orders
Seasonality	False alert	Same-weekday baseline dan minimum sample size
Token leakage	Security incident	Env secret, .gitignore, no token in logs/screenshots

10.3 Suggested MVP sequence

- MVP-A: scope inspection + query-only client + order metric.
- MVP-B: DuckDB snapshots + baseline + revenue anomaly.
- MVP-C: inventory risk + multi-signal diagnosis.
- MVP-D: Streamlit dashboard + alert feed + synthetic demo scenario.
- Extension: returns, payouts, advanced statistical/ML anomaly detection.

Recommended project framing

Build a read-only commerce observability system that detects abnormal store behavior, identifies likely causes from Shopify data, and surfaces evidence-based alerts without modifying the merchant store.

Referensi Teknis

[1] Shopify API access scopes. Referensi scope read/write, termasuk read_reports, read_orders, read_returns, dan read_all_orders permission notes.

<https://shopify.dev/docs/api/usage/access-scopes>

[2] ShopifyQL - latest. Dokumentasi current ShopifyQL; pada September 2026 latest stable ditampilkan sebagai 2026-07.

<https://shopify.dev/docs/api/shopifyql/latest>

[3] shopifyqlQuery. Requirement read_reports dan response tabular ShopifyQL.

<https://shopify.dev/docs/api/admin-graphql/latest/queries/shopifyqlQuery>

[4] Shopify API limits. Calculated query cost, throttle model, dan recommendation untuk caching/retry.

<https://shopify.dev/docs/api/usage/limits>

[5] HTTPX Clients. Connection pooling dan reusable HTTP client.

<https://www.python-httpx.org/advanced/clients/>

[6] HTTPX Timeouts. Timeout management untuk network client.

<https://www.python-httpx.org/advanced/timeouts/>

[7] DuckDB Python API. Embedded analytical database dan integrasi Python.

<https://duckdb.org/docs/stable/clients/python/overview>

[8] DuckDB + Pandas. Import/query Pandas DataFrames pada DuckDB.

https://duckdb.org/docs/current/guides/python/import_pandas

[9] Pandas Rolling Mean. Rolling baseline calculation.

<https://pandas.pydata.org/docs/reference/api/pandas.api.typing.Rolling.mean.html>

[10] Tenacity. Retry dan exponential backoff utilities.

<https://tenacity.readthedocs.io/en/stable/>

[11] Streamlit. Python data app framework untuk dashboard exploration.

<https://docs.streamlit.io/>

[12] Plotly Python. Interactive visualization untuk dashboard.

<https://plotly.com/python/getting-started/>

Source note

Blueprint ini juga menggunakan daftar access scope Shopify yang diberikan pada materi awal project. Scope aktual tetap harus diverifikasi terhadap token/app installation sebelum implementasi.